

Network Compression (二): 多视角 压缩神经网络

Knowledge Distillation, Quantization, Efficient Architecture,
and Dynamic Computation



基于李宏毅老师《机器学习 2025》课程内容整理
2026 年 6 月 9 日

视频信息

频道: 李宏毅深度学习课堂 (Bilibili)

时长: 约 56 分 46 秒

链接: <https://www.bilibili.com/video/BV1TAtwzTE1S?p=52>

目录

1 从 Pruning 到多路线压缩	3
1.1 本章小结	4
2 Knowledge Distillation: 把大模型的行为教给小模型	4
2.1 从单个 teacher 到 ensemble teacher	5
2.2 Temperature: 让 softmax 变得更软	5
2.3 本章小结	6
3 Parameter Quantization: 用更少 bit 表示参数	6
3.1 Binary Weights: 极端的量化	7
3.2 本章小结	9
4 Architecture Design: 直接设计更省的网络	9
4.1 标准卷积的参数量	9
4.2 Depthwise Separable Convolution	10
4.3 Low-Rank Approximation	11
4.4 本章小结	12
5 Dynamic Computation: 按需求改变计算量	12
5.1 Dynamic Depth: 中途出口	13
5.2 Dynamic Width: 可切换宽度	14
5.3 Computation Based on Sample Difficulty	15
5.4 本章小结	16
6 总结与延伸	16
6.1 本章小结	17

1 从 Pruning 到多路线压缩

上一讲聚焦 network pruning：先训练一个较大的模型，再移除低重要性的权重、神经元或结构块，并通过 fine-tuning 恢复性能。本讲继续讨论 network compression，但不再只看 pruning，而是从多个面向压缩神经网络：让小模型模仿大模型、用更少 bit 表示参数、直接设计高效架构，以及让模型按实际需求动态改变计算量。

课程最后的总结页把这一讲的范围列成五个主题：

- **Network Pruning**：从结构中删除冗余部分。
- **Knowledge Distillation**：把大模型或 ensemble 的行为转移给小模型。
- **Parameter Quantization**：用较少 bit 表示权重或 activation。
- **Architecture Design**：重新设计参数效率更高、计算更省的网络模块。
- **Dynamic Computation**：不同输入、设备或状态使用不同计算量。



Concluding Remarks

- Network Pruning
- Knowledge Distillation
- Parameter Quantization
- Architecture Design
- Dynamic Computation

图 1: 本讲最后回顾的 compression 路线：pruning、distillation、quantization、architecture design 与 dynamic computation。¹

这些路线不是互斥的。实际系统中常见组合是：先选一个更高效的 backbone，再用大模型 distillation 训练它，随后做 quantization，必要时再剪枝或加入 early exit。也就是说，压缩不是某个单一技巧，而是围绕部署约束组织的一组工程与学习方法。

压缩的目标不是“越小越好”

模型压缩真正优化的是多目标 trade-off：准确率、模型大小、延迟、吞吐、内存访问、能耗、硬件支持、训练成本与维护复杂度。一个方法若只减少参数量，却没有减少实际 wall-clock latency，在部署上未必有价值。

¹ 视频画面时间区间：00:56:15–00:56:30。

1.1 本章小结

Network compression 可以从结构、数值表示、学习目标、网络模块和运行路径几个层面同时入手。P51 讲的是 pruning，本讲补上其它常见方向，让压缩从“删掉一些东西”扩展成“重新组织模型的知识、表示和计算”。

2 Knowledge Distillation: 把大模型的行为教给小模型

Knowledge Distillation 的基本设定是：先有一个能力较强但较大的 teacher network，再训练一个较小的 student network，让 student 不只学习真实标签，还学习 teacher 对输入的完整输出分布。teacher 可以是单个大模型，也可以是多个模型 ensemble 的平均输出。

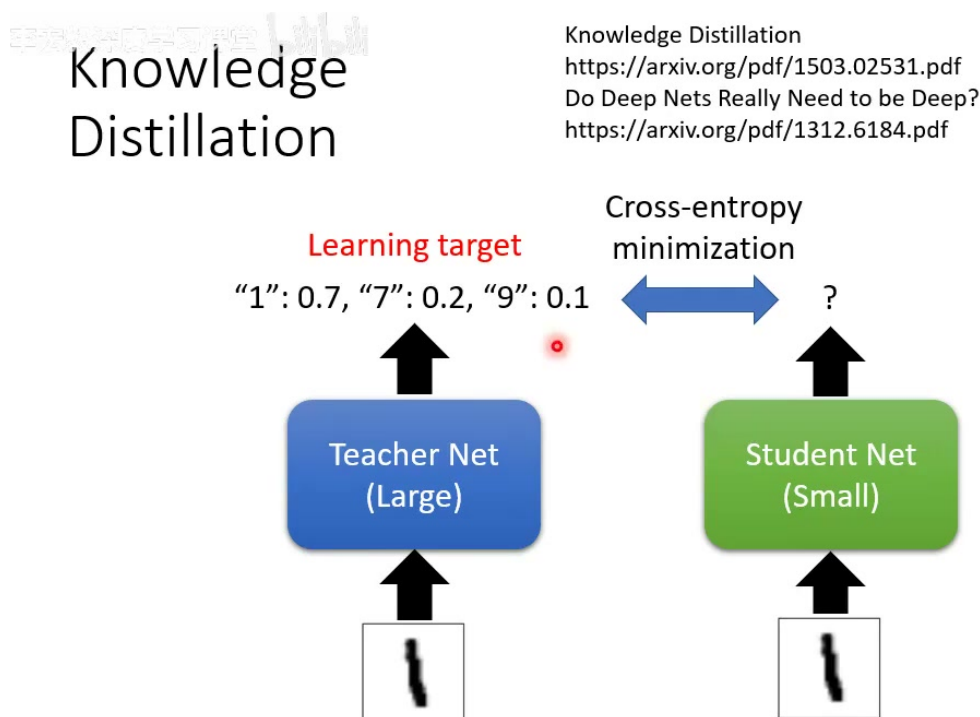


图 2: Knowledge distillation 的核心流程: teacher net 对输入图片给出软目标, 例如“1”的概率为 0.7、“7”为 0.2、“9”为 0.1, student net 通过 cross-entropy 或 KL divergence 去匹配这个分布。²

传统监督学习通常只给 one-hot 标签。比如某张图的标签是数字“1”，训练目标就是 $y_1 = 1$ ，其它类别为 0。但 teacher 的输出分布更细：它可能认为这张图确实最像“1”，但也有一点像“7”或“9”。这种“次高概率”的信息常被称为 **dark knowledge**：它不在硬标签中显式出现，却表达了类别之间的相似性和 teacher 学到的判别边界。

若 teacher 在 temperature T 下输出分布 $q_{\theta_t}^T(y | x)$, student 在同一 temperature 下输出 $p_{\theta_s}^T(y | x)$, 蒸馏损失常写成：

$$L_{KD} = T^2 \text{KL}(q_{\theta_t}^T(\cdot | x) \| p_{\theta_s}^T(\cdot | x)).$$

实际训练中也常把它和真实标签的监督损失相加：

$$L = \alpha L_{\text{hard}}(y, p_{\theta_s}) + (1 - \alpha) L_{KD},$$

其中 L_{hard} 是普通 cross-entropy, α 控制硬标签与 teacher soft label 的权重。

²视频画面时间区间：00:02:15–00:02:30。

为什么 soft label 比 hard label 更丰富

如果 hard label 只告诉 student “这是 1”，student 无法知道它和 7、9 的相似程度。teacher 的 soft label 则告诉 student：这张图主要像 1，但局部笔画也可能使它有一点像 7 或 9。对小模型来说，这种相似度结构能减少盲目探索，也能把大模型的 inductive bias 转移过来。

2.1 从单个 teacher 到 ensemble teacher

teacher 不一定是一张网络。课程图中也给出 ensemble 版本：把很多网络的输出取平均，再拿平均后的分布训练 student。ensemble 往往比单模型更稳健，因为不同模型会犯不同错误，平均后可以降低方差。问题是 ensemble 部署成本高：推理时要跑很多模型，参数量和延迟都会成倍增加。

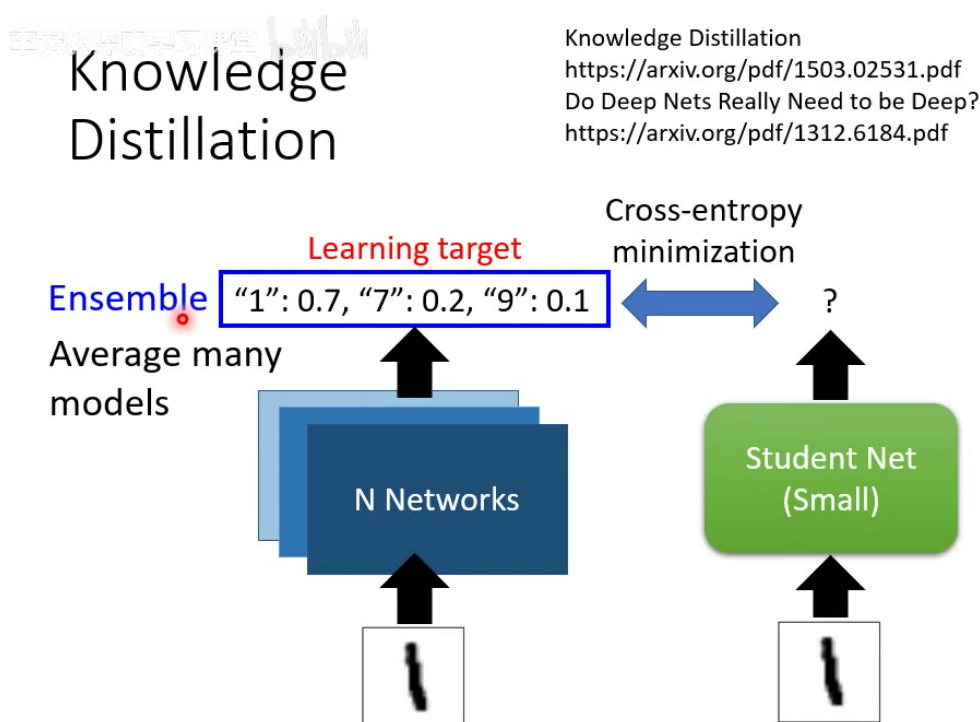


图 3: Ensemble distillation: 先平均多个 teacher network 的输出分布，再让一个小 student 学习这个平均分布，从而把 ensemble 的预测行为压缩到单个模型中。³

Distillation 因此可以理解为一种“行为压缩”：不是直接把 teacher 的参数复制给 student，而是把 teacher 的 input-output function 通过数据和目标分布转移出去。它和 pruning 的精神也有相通处：都先依赖一个较大的模型提供能力或知识，再把可部署模型变小。

2.2 Temperature: 让 softmax 变得更软

softmax 对 logit 的尺度很敏感。如果 logit 差距很大，普通 softmax 会把分布压成接近 one-hot，次要类别的概率几乎为零，dark knowledge 就被隐藏掉了。temperature 的做法是先把 logit 除以 T ：

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

当 $T > 1$ 时，类别之间的 logit 差距被缩小，softmax 输出更平滑；当 T 很大时，原本近乎 one-hot 的分布会显露出更多相对关系。

³视频画面时间区间：00:07:15–00:07:30。

Knowledge Distillation

- Temperature for softmax

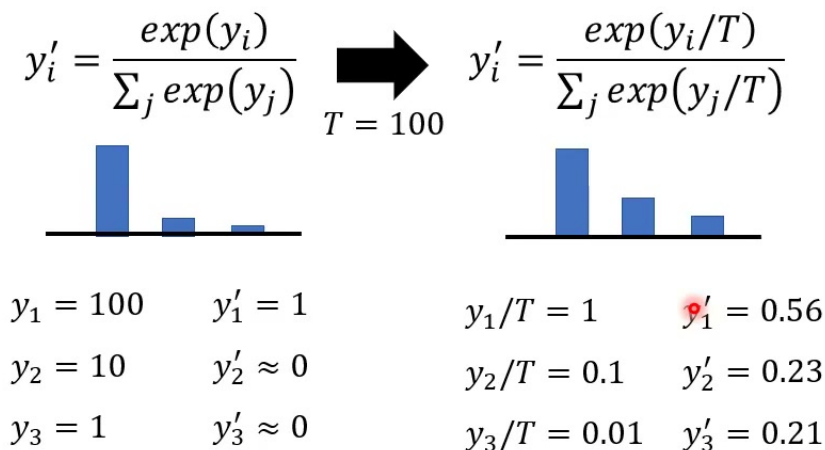


图 4: Temperature softmax 的效果: 原本 $z = (100, 10, 1)$ 会给出接近 one-hot 的分布; 除以 $T = 100$ 后, logit 变成 $(1, 0.1, 0.01)$, 输出约为 $(0.56, 0.23, 0.21)$, 次要类别信息被保留下来。⁴

训练 student 时通常用较高 temperature 计算 teacher 和 student 的 soft distribution; 推理时 student 仍可用普通 $T = 1$ 的输出。这里要注意, temperature 不是为了让模型“更不确定”, 而是为了让训练信号包含类别间关系。若 teacher 本身很差, 或者 soft target 被过度平滑, student 也可能学到噪声。

Distillation 不是免费的性能保证

小模型容量有限, 无法完整复制大模型。蒸馏是否有效取决于 student 架构、teacher 质量、训练数据覆盖、temperature、硬标签与软标签损失权重等因素。如果 student 太小, soft label 可能只能提供有限帮助; 如果 teacher 的偏差很强, student 也会继承这些偏差。

2.3 本章小结

Knowledge distillation 用 teacher 的输出分布训练 student, 把大模型或 ensemble 的行为压缩到小模型中。soft label 的价值在于保留类别相似性, temperature 则让这些相似性更容易被 student 看到。它压缩的不是参数本身, 而是模型学到的函数行为。

3 Parameter Quantization: 用更少 bit 表示参数

Parameter Quantization 关注的是数值表示。神经网络中大量权重通常用 32-bit 浮点数存储, 但很多任务并不需要每个参数都保留这么高的精度。如果能用 16-bit、8-bit, 甚至更少 bit 表示权重, 就能减少模型文件大小和内存带宽需求; 若硬件支持低精度计算, 还可能降低延迟和能耗。

课程先给出最直接的想法: **using less bits to represent a value**。例如把连续权重映射到有限 codebook, 实际存储时只存 codebook index。若 codebook 有 K 个中心, 只需要 $\lceil \log_2 K \rceil$ bit 表

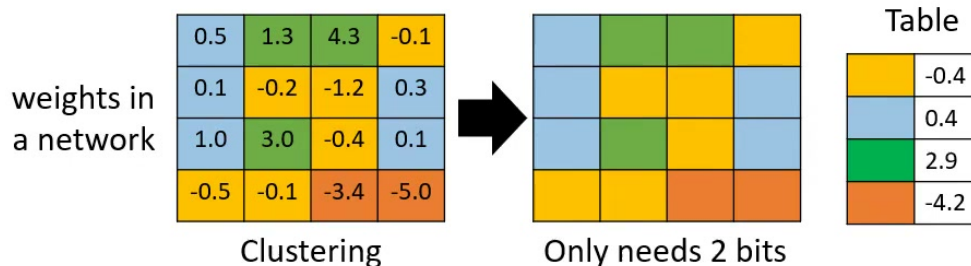
⁴视频画面时间区间: 00:11:00-00:11:15。

示一个权重，再额外保存一个很小的查表。



Parameter Quantization

- 1. Using less bits to represent a value
- 2. Weight clustering



- 3. Represent frequent clusters by less bits, represent rare clusters by more bits
 - e.g. Huffman encoding

图 5: Weight clustering 的量化方式: 把原始权重聚成少数几类, 矩阵中每个位置只保存 cluster index; 若只有 4 个 cluster, 每个位置只需 2 bit, 再用 table 还原代表值。频繁 cluster 还可以用 Huffman encoding 给更短码。⁵

图中把多个实数权重映射成四种颜色, 每种颜色对应一个代表值。这样做有两个层次的压缩:

- **聚类压缩**: 参数本身不再保存完整浮点数, 只保存 cluster id。
- **熵编码压缩**: 高频 cluster 用更短 bit pattern, 低频 cluster 用更长 bit pattern, 例如 Huffman encoding。

量化可以发生在训练后, 也可以发生在训练中。训练后量化 (post-training quantization) 实现简单, 但若精度降得太狠, 模型可能无法适应。量化感知训练 (quantization-aware training) 会在训练时模拟量化误差, 让模型提前学会在低精度约束下工作。

量化要同时看存储、带宽和硬件 kernel

参数变成 8-bit 并不自动意味着推理加速。如果计算图中仍频繁反量化回浮点, 或者目标硬件没有高效的 int8 kernel, 实际速度可能提升有限。量化最稳定的收益通常来自模型文件更小、内存带宽更低; 延迟收益需要结合硬件和框架验证。

3.1 Binary Weights: 极端的量化

Binary weights 是更激进的量化: 权重只允许 +1 或 -1。这样模型存储极小, 乘法也可能简化为符号操作。课程中提到 BinaryConnect、BinaryNet、XNOR-Net 等工作, 它们尝试在训练或推理时使用二值权重。

BinaryConnect 的训练思路可以概括为: 前向传播和反向传播中用二值权重计算网络行为和梯

⁵ 视频画面时间区间: 00:21:00-00:21:15。

度方向，但参数更新仍维护一份 real-valued weights。也就是说，二值网络负责“参与计算”，实数权重负责“累计更新”。这可以避免权重只有 ± 1 时更新空间过于僵硬。

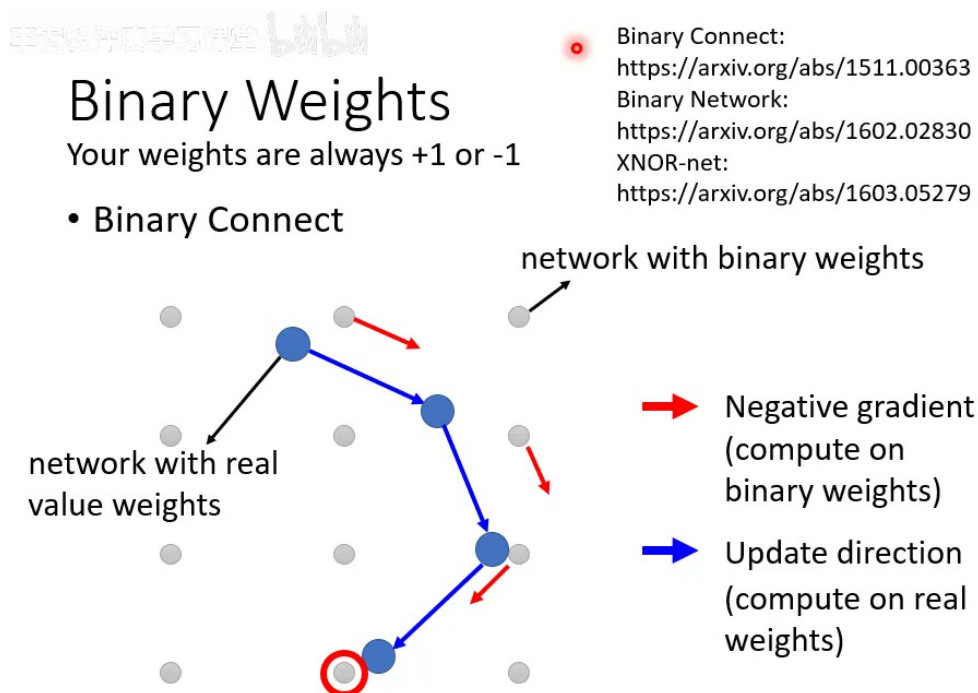


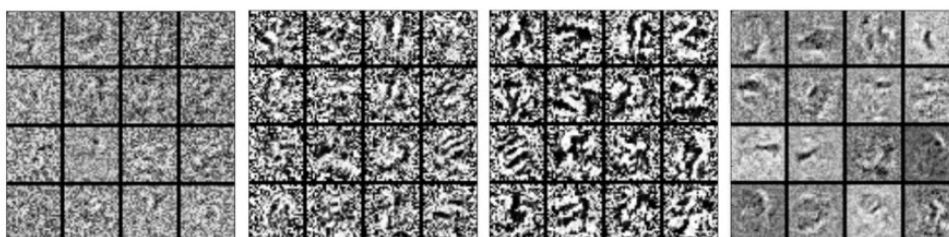
图 6: BinaryConnect 的训练机制示意: network with binary weights 负责计算负梯度, 真实值权重根据更新方向移动; 训练时保留 real-valued weights, 使用时可二值化。⁶

课程也展示了 BinaryConnect 在 MNIST、CIFAR-10、SVHN 上的实验表。表格中 BinaryConnect 的误差率并未灾难性上升, 甚至某些设置下接近或优于没有 regularizer 的 baseline。这说明在过参数化网络中, 很多任务对权重精度并没有想象中那么敏感。

⁶ 视频画面时间区间: 00:22:15–00:22:30。

Binary Weights

Method	MNIST	CIFAR-10	SVHN
No regularizer	$1.30 \pm 0.04\%$	10.64%	2.44%
BinaryConnect (det.)	$1.29 \pm 0.08\%$	9.90%	2.30%
BinaryConnect (stoch.)	$1.18 \pm 0.04\%$	8.27%	2.15%
50% Dropout	$1.01 \pm 0.04\%$		



<https://arxiv.org/abs/1511.00363>

图 7: Binary weights 的实验结果示意: BinaryConnect 在多个数据集上能达到接近 baseline 的表现, 显示出神经网络对低精度参数的容忍度。⁷

不过, 二值化是非常强的约束。它更适合特定架构、特定硬件和特定任务; 对大型语言模型、多模态模型或高精度回归任务, 直接二值化往往会带来明显损失。现代部署中更常见的是 int8、int4、混合精度量化, 以及对不同层使用不同 bit 的方法。

3.2 本章小结

Parameter quantization 通过降低数值精度压缩模型。Weight clustering 把连续权重映射到 codebook, binary weights 则把这种思想推到极端。量化的关键不只是“bit 数变少”, 还包括是否能训练稳定、是否能保持精度、是否能被目标硬件真正利用。

4 Architecture Design: 直接设计更省的网络

Pruning 和 quantization 都是在已有模型上做压缩; architecture design 则从一开始就设计更高效的模块。课程用 convolution 为例说明: 标准卷积的参数量和计算量往往很大, 而适当拆解卷积结构, 可以在保留表达力的同时显著减少参数。

4.1 标准卷积的参数量

设输入 feature map 有 I 个 channel, 输出有 O 个 channel, 卷积核大小为 $k \times k$ 。标准 convolution 中, 每个 output channel 都需要一个覆盖所有 input channel 的卷积核, 因此参数量为

$$k \times k \times I \times O.$$

课程图中例子为 $k = 3$ 、 $I = 2$ 、 $O = 4$, 所以参数量是

$$3 \times 3 \times 2 \times 4 = 72.$$

⁷视频画面时间区间: 00:23:30–00:23:45。

Review: Standard CNN

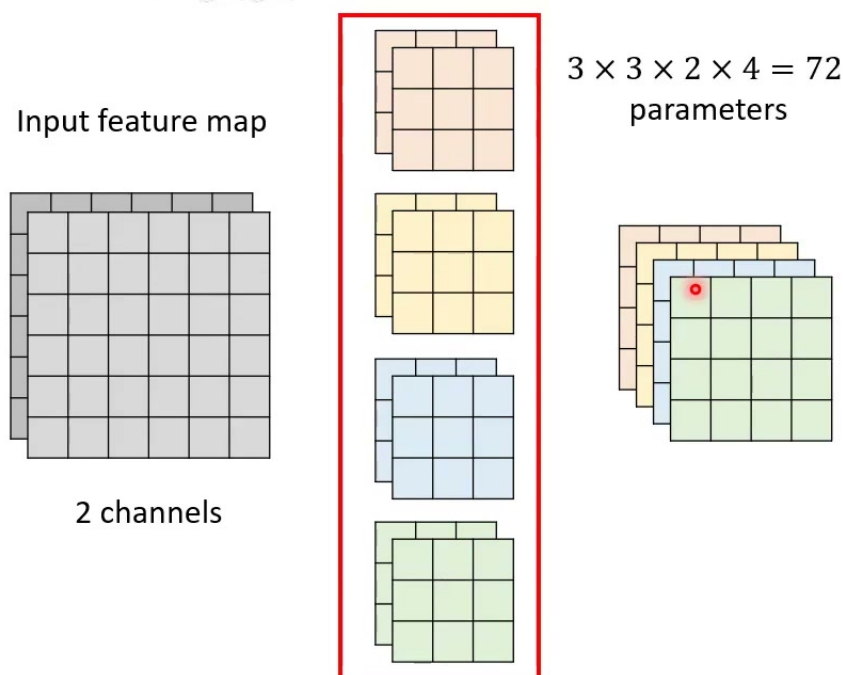


图 8: 标准 CNN 卷积参数量: 每个 output channel 都连接所有 input channel, 因此参数量为 k^2IO ; 图中 $3 \times 3 \times 2 \times 4 = 72$ 。⁸

标准卷积的问题不是它“不好”，而是它同时混合了两类计算：空间维度上的局部特征抽取，以及 channel 之间的信息混合。Depthwise separable convolution 的想法就是把这两件事拆开做。

4.2 Depthwise Separable Convolution

Depthwise separable convolution 分为两步：

1. **Depthwise convolution:** 每个 input channel 各自使用一个 $k \times k$ filter, 只在本 channel 内做空间卷积, 参数量为 k^2I 。
2. **Pointwise convolution:** 用 1×1 convolution 混合 channel, 产生 O 个 output channel, 参数量为 IO 。

总参数量从标准卷积的 k^2IO 变为

$$k^2I + IO.$$

两者比例为

$$\frac{k^2I + IO}{k^2IO} = \frac{1}{O} + \frac{1}{k^2}.$$

当 $k = 3$ 且 O 较大时, 比例约为 $1/9$ 加一个很小的项, 参数和计算都可明显下降。

⁸视频画面时间区间: 00:30:30-00:30:45。

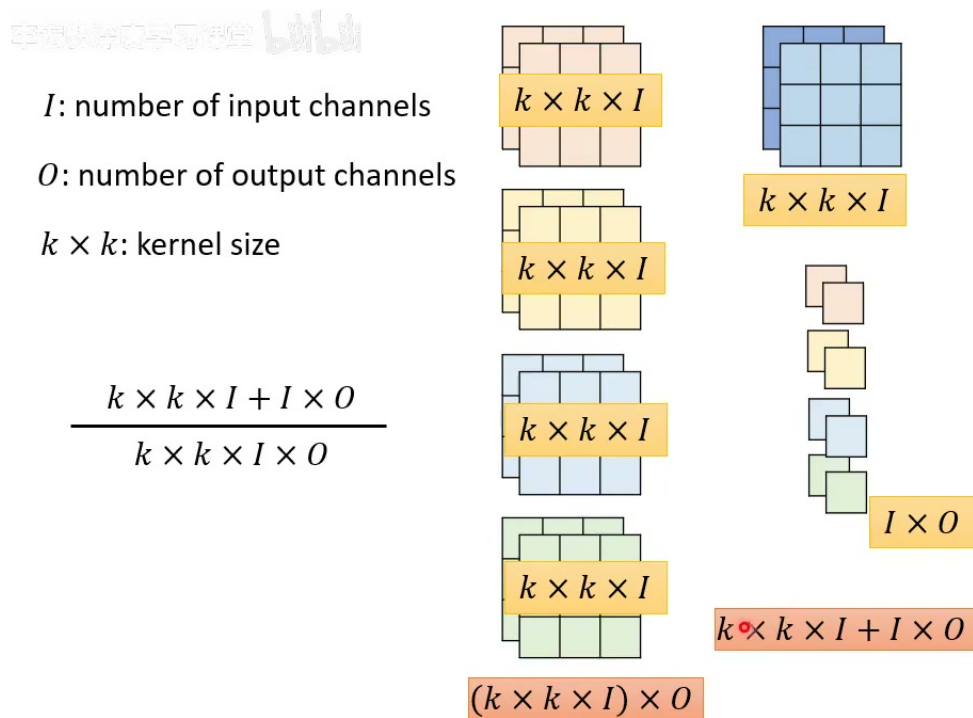


图 9: Depthwise separable convolution 的参数量对比: 标准卷积为 $k^2 IO$, 拆成 depthwise 与 pointwise 后为 $k^2 I + IO$, 比例为 $\frac{1}{O} + \frac{1}{k^2}$ 。⁹

Depthwise separable convolution 是 MobileNet、Xception 等高效 CNN 架构的重要基础。它牺牲的是“每个空间 filter 直接跨所有 channel 混合”的自由度, 换来更低参数数量和更少计算。现代高效网络往往会在这种模块基础上继续加入 inverted bottleneck、channel shuffle、squeeze-and-excitation 等设计。

模块设计也是一种压缩

如果一个架构从训练开始就使用更高效的模块, 它不需要先训练一个笨重模型再压缩。它把压缩目标写进了模型的 inductive bias: 哪些交互值得保留, 哪些交互可以分解, 哪些计算可以共享。

4.3 Low-Rank Approximation

Low-rank approximation 是另一类常见结构压缩。若全连接层权重矩阵 $W \in \mathbb{R}^{M \times N}$ 近似低秩, 可以把它分解为两个较小矩阵:

$$W \approx UV, \quad U \in \mathbb{R}^{M \times K}, \quad V \in \mathbb{R}^{K \times N}, \quad K < \min(M, N).$$

原本参数量是 MN , 分解后是 $MK + KN$ 。只要 K 足够小, 参数就会明显减少。从网络结构角度看, 这相当于在原本一层 linear mapping 中插入一个 bottleneck hidden dimension。

⁹视频画面时间区间: 00:37:15–00:37:30。

Low rank approximation

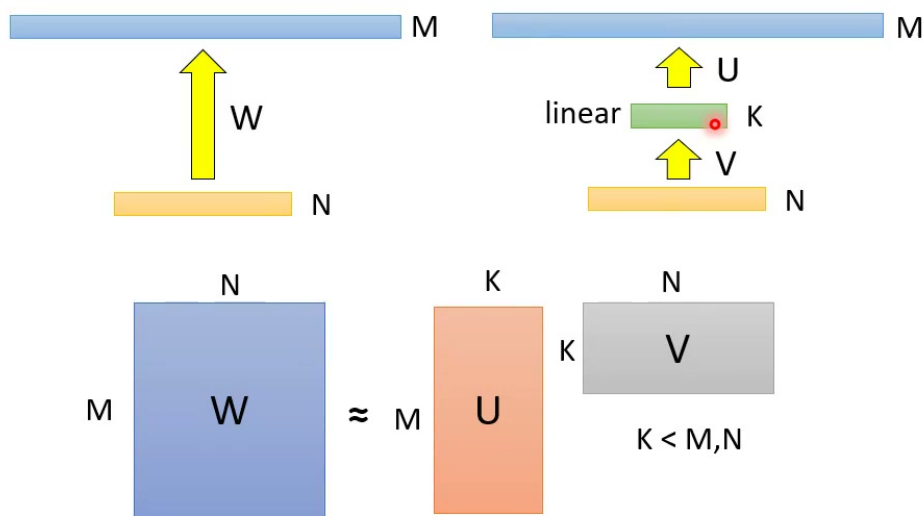


图 10: Low-rank approximation: 把大矩阵 W 近似为 U 与 V 的乘积, 中间 rank K 小于 M, N , 从而用两个小矩阵替代一个大矩阵。¹⁰

Low-rank 思想也可以用于 convolution: 把一个大卷积分解成多个较小卷积, 或者把 channel mixing 与 spatial filtering 拆开。与 pruning 相比, low-rank 分解得到的结构通常更规整, 更容易被常规矩阵乘法或卷积 kernel 利用。

4.4 本章小结

Architecture design 从模型结构本身降低计算量。标准卷积的 k^2IO 参数可以通过 depthwise separable convolution 降到 $k^2I + IO$; 大矩阵也可以用 low-rank factorization 近似。它们共同说明: 压缩不只是在训练后删参数, 也可以在设计阶段改变网络的计算分解方式。

5 Dynamic Computation: 按需求改变计算量

前面几类方法通常得到一个固定大小、固定计算图的模型。Dynamic computation 则进一步问: 是否每次推理都必须运行同样多的 layer、channel 或 block? 现实中, 不同设备、不同电量状态、不同输入难度, 可能需要不同计算量。

¹⁰ 视频画面时间区间: 00:41:00–00:41:15。

Dynamic Computation

- The network adjusts the computation it need.



图 11: Dynamic computation 的动机：模型可以根据设备类型、电量状态或任务需求调整计算量，而不是对所有情境都运行同一套固定计算。¹¹

一个朴素办法是准备多个模型：手机低电量用小模型，云端或高电量用大模型。但这样会增加训练、存储和维护成本。Dynamic computation 希望用一个模型覆盖多种计算预算，让它在运行时选择深度、宽度或路径。

5.1 Dynamic Depth: 中途出口

Dynamic depth 在网络中加入多个 early exit。浅层输出 y' 、中层输出 y'' 、最终输出 y 都可以接受监督。训练时把每个出口的误差加起来：

$$L = e_1 + e_2 + \dots + e_L.$$

这样浅层出口也学会做预测。推理时，如果资源有限或中途预测已经足够可信，就可以提前停止；如果需要更高精度，则继续往后跑。

¹¹ 视频画面时间区间：00:47:15–00:47:30。

Dynamic Depth

$$L = e_1 + e_2 + \dots + e_L$$

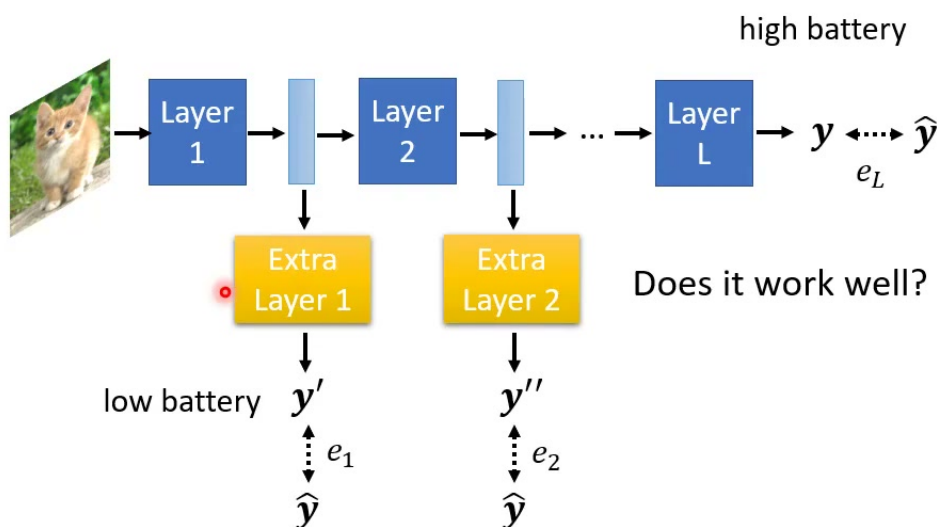


图 12: Dynamic depth: 在中间层接出 extra layers 形成多个出口, 低预算时可以使用较浅输出, 高预算时继续通过完整网络; 训练损失可写为各出口误差之和。¹²

Dynamic depth 的关键难点是出口判定。若模型太早退出, 可能因为浅层特征不足而犯错; 若总是不退出, 又失去节省计算的意义。常见做法包括根据 confidence、entropy、margin 或 learned controller 判断是否继续。

5.2 Dynamic Width: 可切换宽度

Dynamic width 让同一个网络支持不同宽度。宽模式使用更多神经元或 channel, 窄模式只使用其中一部分。课程图中给出 Slimmable Neural Networks: 同一网络可以对应多种宽度, 每一种宽度都有自己的输出和误差项:

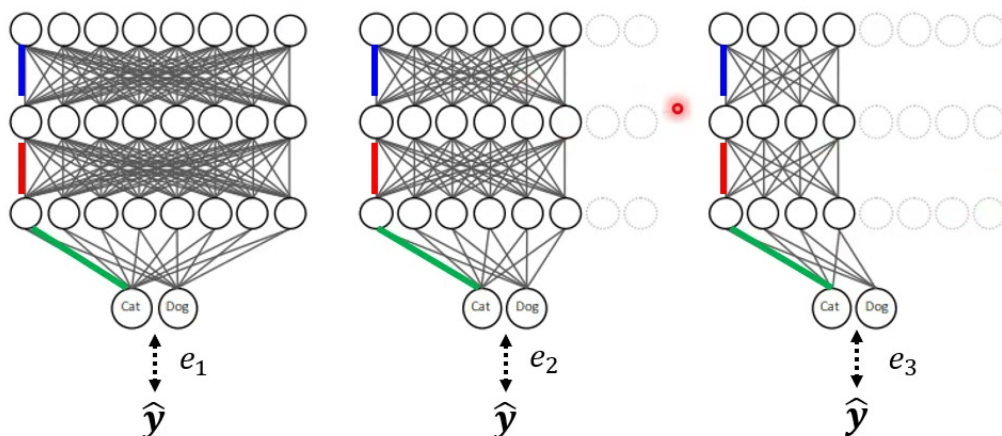
$$L = e_1 + e_2 + e_3.$$

训练时同时约束多个宽度, 使子网络也能独立工作。

¹² 视频画面时间区间: 00:50:45-00:51:00。

Dynamic Width

$$L = e_1 + e_2 + e_3$$



Slimmable Neural Networks
<https://arxiv.org/abs/1812.08928>

图 13: Dynamic width: 同一网络共享参数, 但可以启用不同数量的神经元或 channel, 形成从宽到窄的多个子网络。¹³

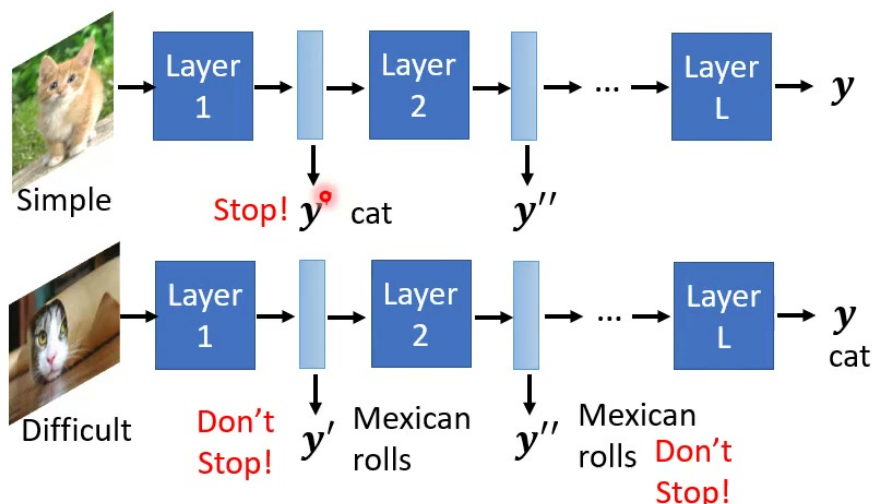
这种方法比维护多套模型更省存储, 但训练更复杂。宽网络和窄网络共享权重, 彼此的梯度可能互相干扰; batch normalization、activation range 和 calibration 也可能随宽度变化。工程实现中通常需要额外技巧保证不同宽度都稳定。

5.3 Computation Based on Sample Difficulty

Dynamic computation 还可以根据样本难度调整。简单样本经过浅层就能正确判断, 可以早停; 困难样本则继续经过更多层。课程图中用两张图说明: 一张简单小猫图在浅层已经预测为 cat, 可以 stop; 另一张较难图浅层误判为 Mexican rolls, 因此不能停, 必须继续往深层处理。

¹³ 视频画面时间区间: 00:52:15–00:52:30。

Computation based on Sample Difficulty



- SkipNet: Learning Dynamic Routing in Convolutional Networks
- Runtime Neural Pruning
- BlockDrop: Dynamic Inference Paths in Residual Networks

图 14: 按样本难度分配计算: 简单样本浅层输出已足够可信时早停; 困难样本若浅层判断错误或不确定, 则继续运行后续层。¹⁴

这类方法的直觉很强: 不是所有输入都同样难。对于图像分类, 清晰、典型的样本可能很早就被识别; 模糊、遮挡、分布外或类别相近的样本则需要更深层特征。课程提到的相关方向包括 SkipNet、Runtime Neural Pruning 与 BlockDrop, 它们都在探索如何为不同输入选择不同推理路径。

Early exit 需要可靠的不确定性估计

如果 confidence 本身不可靠, 模型可能对错误答案非常自信, 从而过早退出。Dynamic computation 在节省计算前, 必须保证退出策略经过校准和验证, 否则会把一部分错误隐藏在“低成本推理”中。

5.4 本章小结

Dynamic computation 让模型在推理时根据设备、资源或样本难度改变计算量。Dynamic depth 改变运行层数, dynamic width 改变启用宽度, sample-dependent routing 则按输入选择路径。它的优势是灵活, 但核心风险是控制策略: 什么时候少算, 什么时候必须多算。

6 总结与延伸

本讲把 network compression 从 pruning 扩展到更完整的技术地图。不同方法压缩的是不同对象:

- **Pruning 压缩结构:** 删除权重、神经元、channel、head 或 block。
- **Distillation 压缩行为:** 用小模型学习大模型或 ensemble 的输出分布。

¹⁴ 视频画面时间区间: 00:55:15–00:55:30。

- **Quantization 压缩表示**：用更少 bit 存储和计算权重、activation。
- **Architecture design 压缩计算分解**：用更高效的模块替代昂贵模块。
- **Dynamic computation 压缩每次推理**：对不同情境使用不同计算预算。

这些方法背后的共同问题是：神经网络中哪些部分是“必须保留的能力”，哪些只是训练便利、数值冗余或固定计算图带来的浪费？Pruning 说冗余连接可以删；distillation 说大模型的行为可以转教给小模型；quantization 说参数不必用高精度表示；efficient architecture 说计算可以重新分解；dynamic computation 说每个样本也不必走同样路径。

部署视角下的推荐思路

真实应用中，通常先明确瓶颈：是模型文件太大、内存带宽太高、延迟太高、能耗太高，还是不同设备需要不同预算。然后再选方法：文件大小优先可先看 quantization；延迟优先要看 architecture 与 structured pruning；精度损失难以接受时可加入 distillation；输入难度差异明显时再考虑 early exit 或 dynamic routing。

从学习角度看，compression 也揭示了深度模型的一个重要事实：训练时需要的大容量，和部署时最终需要的容量，不一定相同。大模型可能是一个强 teacher、一个优化过程中的宽搜索空间，或一个可被提炼的知识载体；压缩方法则尝试把真正有用的部分留下来，让模型在现实约束下运行。

6.1 本章小结

P52 的核心不是某个单独技巧，而是 compression 的系统视角：参数、结构、数值、知识与运行路径都可以被压缩。后续真正做模型部署时，应把这些方法当作工具箱，根据任务、硬件和误差预算组合使用，而不是只追求某一个压缩率数字。